

RECURSO COMPLEMENTARIO

Fundamentos de Transformers

Nota técnica 14

Entrenamiento autoregresivo

Curso abierto

2026-I

Entrenar un modelo autoregresivo es ajustar parámetros para aumentar la probabilidad promedio del siguiente token observado.

1 Ciclo de optimización

La arquitectura produce logits; el entrenamiento define cómo modificarlos a través de parámetros. El ciclo mínimo es

$$\text{batch} \rightarrow \text{forward} \rightarrow \text{logits} \rightarrow \mathcal{L} \rightarrow \nabla_{\theta} \mathcal{L} \rightarrow \theta' \rightarrow \text{validación}.$$

Esta cadena se lee así: se toma un batch, se calculan predicciones, se mide error, se retropropaga gradiente, se actualizan parámetros y se evalúa en datos no usados para actualizar.

Idea clave 1 (Objetivo). El modelo no aprende porque genera texto durante entrenamiento. Aprende porque la pérdida empuja sus logits a asignar más probabilidad a los targets observados.

Ampliación conceptual

El entrenamiento une tres objetos: datos, modelo y objetivo. Los datos aportan pares X/Y ; el modelo produce logits; el objetivo define cómo convertir esos logits en una señal escalar de error. Sin cualquiera de estos tres elementos, no hay aprendizaje supervisado autoregresivo.

La cross-entropy es sensible a la probabilidad asignada al token observado, no solo a si el token más probable coincide. Por eso un modelo puede mejorar su pérdida aunque no cambie la predicción greedy en algunos casos: basta con redistribuir mejor la probabilidad.

El paso de optimización depende del batch. El gradiente calculado es una estimación local de cómo cambiar parámetros para reducir la pérdida en esos ejemplos. Con batches distintos, el gradiente puede variar. El entrenamiento completo es una acumulación de muchas actualizaciones parciales.

2 Batch autoregresivo

Con entradas $X \in \mathbb{N}^{B \times T}$ y targets $Y \in \mathbb{N}^{B \times T}$, cada $Y_{b,t}$ corresponde al siguiente token de $X_{b,t}$. El modelo produce

$$Z = f_{\theta}(X) \in \mathbb{R}^{B \times T \times V}.$$

Para cada posición, softmax sobre el eje V define

$$p_{\theta}(Y_{b,t} \mid X_{b,\leq t}).$$

3 Cross-entropy

La pérdida media es

$$\mathcal{L}(\theta) = -\frac{1}{BT} \sum_{b=1}^B \sum_{t=1}^T \log p_{\theta}(Y_{b,t} \mid X_{b,\leq t}).$$

Esta ecuación se lee así: se penaliza al modelo en cada posición si asigna baja probabilidad al token observado.

En implementación, los logits suelen reorganizarse de

$$(B, T, V) \rightarrow (BT, V),$$

y los targets de

$$(B, T) \rightarrow (BT).$$

Esto no cambia el problema matemático; solo presenta todas las posiciones como ejemplos de clasificación sobre V clases.

4 Gradiente y actualización

La retropropagación calcula

$$\nabla_{\theta} \mathcal{L}(\theta),$$

y un optimizador actualiza parámetros, por ejemplo de forma idealizada:

$$\theta' = \theta - \eta \nabla_{\theta} \mathcal{L}(\theta).$$

Esta ecuación se lee así: los parámetros se mueven en una dirección que reduce la pérdida local bajo el batch y el optimizador elegidos.

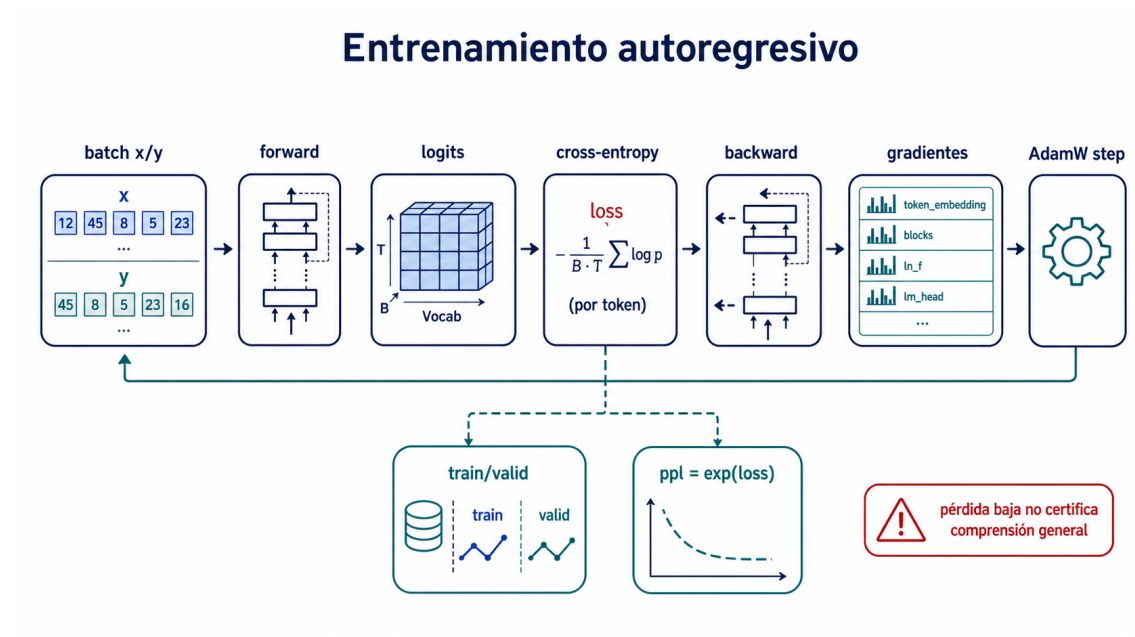
5 Validación y perplexity

La validación calcula la misma pérdida sin actualizar parámetros. La perplexity es

$$\text{PPL} = \exp(\mathcal{L}).$$

Debe interpretarse como cambio de escala de la entropía cruzada, no como medida directa de comprensión.

6 Figura conceptual



La figura presenta el ciclo de entrenamiento: datos, forward, pérdida, backward, actualización y validación. La separación entre entrenamiento y validación es parte de la evidencia experimental.

Cómo leer la figura

La figura debe leerse como un circuito con una única etapa que cambia parámetros: el paso del optimizador. Forward y loss calculan valores; backward calcula gradientes; validación mide sin modificar.

La separación train/valid no es formalidad. Permite detectar si el modelo mejora solo sobre ejemplos vistos o si mantiene desempeño en datos retenidos.

7 Límites de interpretación

Una pérdida menor indica mejor ajuste promedio al corpus y configuración evaluados. No prueba generalización amplia, factualidad ni razonamiento. En corpus pequeños, una brecha creciente entre entrenamiento y validación puede sugerir sobreajuste.

La perplexity facilita comparar pérdidas en escala exponencial, pero puede ser engañosa si se comparan tokenizadores, vocabularios o corpus distintos. Una perplexity menor bajo un vocabulario no siempre equivale a mejor modelo bajo otro.

Observación 1 (Validación). Validar sin gradiente evita modificar el modelo durante la medición. Si la validación actualiza parámetros, deja de ser una medición independiente.

8 Resumen

Resumen 1 (Cierre de la nota). El entrenamiento autoregresivo convierte logits en pérdida, pérdida en gradientes y gradientes en actualización de parámetros. La forma (B, T, V) de los logits y el desplazamiento X/Y conectan la arquitectura con la cross-entropy por token.

9 Preguntas de estudio

1. ¿Por qué los targets se desplazan una posición respecto a las entradas?
2. ¿Qué significa reorganizar logits de (B, T, V) a (BT, V) ?
3. ¿Qué cambia durante backward y qué cambia durante el paso del optimizador?
4. ¿Por qué la validación debe calcularse sin gradiente?
5. ¿Por qué perplexity no equivale a comprensión?

10 Continuidad

Una vez entrenado, el modelo puede usarse para generar tokens iterando distribuciones locales y aplicando estrategias de muestreo.